SQL Practical Tasks

Constraints (Tasks 1–6)

01 Constraints	**Build a Users table with NOT NULL & UNIQUE** Create a users table where username and email are both NOT NULL and UNIQUE. Insert a valid row, then try inserting a duplicate username and observe the error.
02 Constraints	**Test NOT NULL enforcement** Using the same users table, try inserting a row with a NULL value for email. Confirm the database rejects it and explain why NOT NULL protects data quality.
03 Constraints	**Link two tables with a FOREIGN KEY** Create a profiles table referencing users(user_id). Insert a valid profile. Then try inserting one with a non-existent user_id to trigger a referential integrity error.
04 Constraints	**Apply a CHECK constraint on price** Create a products table where price must be greater than 0. Try inserting a product with a negative price (-10.00) and observe the CHECK violation error. **Hint:** `price DECIMAL(10,2) CHECK (price > 0)`
05 Constraints	**Use DEFAULT values in action** Add stock INT DEFAULT 0 and category VARCHAR(50) DEFAULT 'General' to products. Insert a row without those columns, then SELECT to verify the defaults were applied automatically.
06 Constraints	**Design a banking schema with all constraints** Create Customers, Accounts, and Transactions tables using PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, CHECK, and DEFAULT together — simulating a real-world banking system.

DDL — Data Definition Language (Tasks 7–10)

07 DDL	**Add a column with ALTER TABLE** Create a staff table with id and name columns. Then use ALTER TABLE to add a city column. Verify the new column appears with DESCRIBE staff.

08 DDL	**Rename a column** Using the staff table, rename the name column to full_name using ALTER TABLE ... RENAME COLUMN. Confirm the change, then insert a row using the new column name.

09 DDL	**TRUNCATE vs DELETE — understand the difference** Insert 3 rows into staff. Use TRUNCATE TABLE to clear all data. Then compare behavior with DELETE FROM staff — note that TRUNCATE keeps the structure but removes rows instantly.

10 DDL	**DROP a table and database safely** Create a practice database keywords_practice, add a table inside it, then clean up using DROP TABLE and DROP DATABASE. Confirm it is gone with SHOW DATABASES.

DML — Data Manipulation Language (Tasks 11–13)

11 DML	**INSERT multiple rows at once** Insert at least 4 employees into a staff table (id, full_name, city) in a single INSERT statement. Then use SELECT * to confirm all rows were added correctly.

12 DML	**UPDATE a specific row** Change the city of the employee with id = 1 to 'Tokyo'. Run a SELECT before and after the UPDATE statement to visually confirm the change was applied.

13 DML	**DELETE a row by name** Delete the employee named 'Charlie' from staff using a WHERE clause. Verify the row is gone. Then try deleting a name that does not exist — observe the 0 rows affected result.

Filtering & Sorting (Tasks 14–18)

14 Filtering	**Filter with WHERE + AND / OR / NOT** From a students table, retrieve students older than 20 whose names do NOT start with 'D'. Combine WHERE, AND, and NOT LIKE in a single query.
15 Filtering	**Use BETWEEN and IN together** Write two separate queries: one using BETWEEN to find students aged 20-25, and another using IN to fetch students with specific IDs (1 and 3). Compare the result sets.
16 Filtering	**Sort and limit results** Retrieve all students sorted by age in descending order. Then add LIMIT 2 to get only the top 2 oldest. Also practice sorting alphabetically with ORDER BY name ASC.
17 Filtering	**Remove duplicates with DISTINCT** Add a course column to students with some duplicate values. Use SELECT DISTINCT course to get a unique list. Compare the output with a plain SELECT course to see the difference.
18 Filtering	**Aggregate with GROUP BY & HAVING** Group students by course and count how many are enrolled in each. Use HAVING to show only courses with more than 1 student. Understand why HAVING is used instead of WHERE here.

DCL — Data Control Language (Tasks 19–20)

19 DCL	**GRANT and REVOKE user permissions** Create a new MySQL user 'john'@'localhost'. Grant SELECT and INSERT privileges on your database. Then revoke INSERT and verify the change with SHOW GRANTS.
20 DCL	**Read-only role simulation** Create user 'alice'@'localhost' and grant only SELECT on a specific table. Try running an INSERT as that user — confirm it is denied. This simulates a real-world read-only analyst role.